Capstone Project

Team members:

1) Navaneeth Kumar

2) Anoushka Aditi Achrya

3) Dyuti A Urs

4) Megha Pradhan

5) KS Hemanth

Project structure:

The Flight Booking System is designed using Microservices Architecture with a clear separation of concerns. Each service is independently deployable and handles a specific business function.

Overall Architecture:

- **Frontend: React Application**
- **Backend: Spring Boot Microservices**
- **API Gateway: Single entry point**
- **Service Registry: Eureka Server**
- **Database: Separate DB for each service**

Frontend structure:

```
src/
├── components/
├── pages/
├── services/   (API calls)
├── routes/
├── context/    (Authentication state)
└── utils/
```

Backend structure:

```
flight-booking-system/
|
├── api-gateway/
├── service-registry/
|
├── user-auth-service/
├── flight-search-service/
├── booking-service/
├── ticket-service/
├── admin-inventory-service/
```

FRONTEND IMPLEMENTATION:

The frontend is developed using React and interacts with backend services through REST APIs.

◆ Key Pages

👤 User Side

- **Login / Register**
- **Flight Search Page**
- **Flight Results Page**
- **Booking Page**
- **Ticket View (PNR based)**
- **Booking History**
- **Cancel Ticket**

🛠️ Admin Side

- **Admin Login**
- **Add Airline**
- **Add Flight Schedule**
- **Manage Inventory**

◆ API Communication

Frontend sends requests to backend via:

```
Frontend → API Gateway → Microservice → Database
```

JWT token is attached in headers for secure requests.

Backend Implementation:

The backend is implemented using Spring Boot Microservices.

◆ Key Technologies

- **Spring Boot**
- **Spring Security**
- **JWT Authentication**
- **API Gateway**
- **REST APIs**

◆ Microservice Responsibilities

1. User & Auth Service

- **User login & registration**
- **JWT token generation**
- **Role-based access**

2. Flight Search Service

- **Fetch flights from database**
- **Filter by date, source, destination**

3. Booking Service (Core Logic)

- **Validate seat availability**
- **Generate PNR**
- **Store booking and passenger details**
- **Handle transactions**

4. Ticket Service

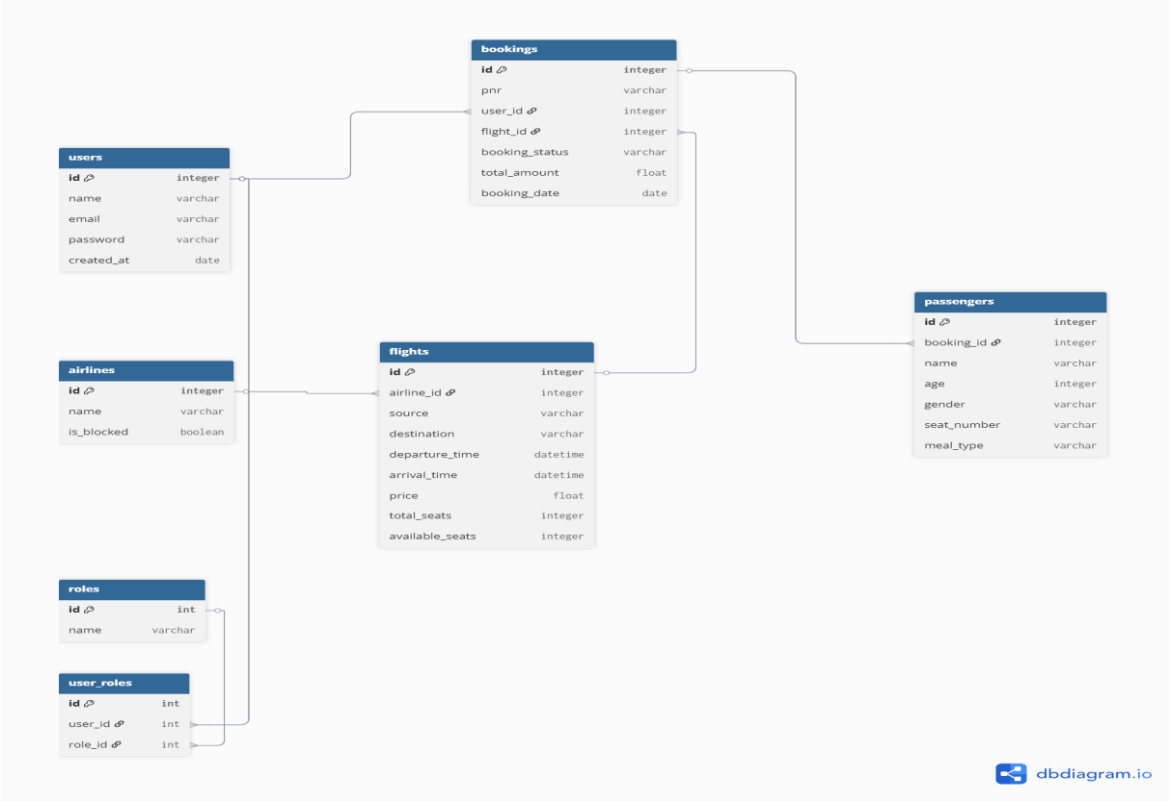
- **Fetch ticket by PNR**
- **Booking history by email**
- **Cancel booking (based on rules)**

5. Admin Service

- **Add airline**
- **Add flight schedules**
- **Manage pricing and seats**

Database Structure:

Each microservice maintains its own database



1. USERS:
id (INT, PK)
name (STRING)
email (STRING, UNIQUE)
password (STRING)
role (STRING - USER/ADMIN)
created_at (DATE)

2. AIRLINES:
id (INT, PK)
name (STRING)
is_blocked (BOOLEAN)

3. FLIGHTS
id (INT, PK)
airline_id (INT, FK)
source (STRING)
destination (STRING)
departure_time (DATETIME)
arrival_time (DATETIME)
price (FLOAT)
total_seats (INT)
available_seats (INT)

4. BOOKINGS
id (INT, PK)
pnr (STRING, UNIQUE)
user_id (INT, FK)
flight_id (INT, FK)
booking_status (STRING)
total_amount (FLOAT)
booking_date (DATE)

5. PASSENGERS
id (INT, PK)

booking_id (INT, FK)
name (STRING)
age (INT)
gender (STRING)
seat_number (STRING)
meal_type (STRING)

RELATIONSHIPS:

Users -> Bookings (1:M)
Airlines -> Flights (1:M)
Flights -> Bookings (1:M)
Bookings -> Passengers (1:M)